4강. React Props 및 useState

React

Props 객체

- **Props 객체**
  - ✓ props(properties)는 컴포넌트를 마치 HTML 태그처럼 사용해 값을 속성 형태로 전달합니다.

강아지 컴포넌트

품종: 말티즈

나이: 3

App.jsx

```
{/* Dog 컴포넌트 사용 - props 전달 */}
<Dog
  breed="말티즈"
  age={3}
/>
```

Props 객체

- **Props 객체**
  - ✓ Dogjsx

```jsx
const Dog = (props) => {
  // props 객체에서 breed와 age를 구조 분해 할당
  const { breed, age } = props;

  return (
    <div>
      <h2>강아지 컴포넌트</h2>
      <p>품종: {breed}</p>
      <p>나이: {age}</p>
    </div>
  )
}

export default Dog
```

Props 객체

- **Props 객체**
  - ✓ Dogjsx

```jsx
const Dog = ({ breed, age }) => {
  return (
    <div>
      <h2>강아지 컴포넌트</h2>
      <p>품종: {breed}</p>
      <p>나이: {age}</p>
    </div>
  )
}
```

Props 객체

- **Props.children**
  - ✓ <Box></Box> 사이의 모든 것이 {children}으로 전달됩니다.

박스 안의 내용

이것은 Box 컴포넌트 안에 있는 내용입니다.

또 다른 박스

이것은 또 다른 Box 컴포넌트 안에 있는 내용입니다.

```
  {/* Box 컴포넌트 사용 - children을 통해 내용 전달 */}
  <Box>
    <h3>박스 안의 내용</h3>
    <p>이것은 Box 컴포넌트 안에 있는 내용입니다.</p>
  </Box>

  <Box>
    <h3>또 다른 박스</h3>
    <p>이것은 또 다른 Box 컴포넌트 안에 있는 내용입니다.</p>
  </Box>
</>
```

Props 객체

- **Props.children**
  - ✓ Box.jsx

```
// Box 컴포넌트 정의
const Box = ({children}) => {
  const boxStyle = {
    width: '200px',
    border: '1px solid black',
    padding: '20px',
    margin: '10px',
  };

  return <div style={boxStyle}>{children}</div>;
}

export default Box
```

프로필 카드 UI 구현

컴포넌트 개요

* **파일명:** `src/card/Profile.jsx`
* **주요 목적:** 프로필 카드 UI 표시 (이미지 + 정보)
* **구성 컴포넌트:** Profile, Avatar, Card

프로필 카드 UI 구현

- Profile.jsx

```
import profilePhoto from '../assets/sudo.jpg'
import Card from './Card'
import Avatar from './Avatar'

export default function Profile() {
  const AVATAR_SIZE = { width: '300', height: '200' };

  return (
    <Card>
      <Avatar
        className="avatar"
        size={AVATAR_SIZE}
        person={{
          name: '김기용',
          imageUrl: profilePhoto
        }}
      />
    </Card>
  );
}
```

프로필 카드 UI 구현

- Avatar.jsx

```
const Avatar = ({ person, size }) => {
  return (
    <img
      className="avatar"
      src={person.imageUrl}
      alt={person.name}
      width={size.width}
      height={size.height}
    />
  );
}

export default Avatar;
```

프로필 카드 UI 구현

- Card.jsx

```
const Card = ({ children }) => {
  return (
    <div className="card">
      {/* 카드 컴포넌트의 내용 - Avatar */}
      {children}
    </div>
  );
}

export default Card;
```

컴포넌트(Component) 상태

- **React Hook: 기본 상태 관리**

리엑트에서는 상태를 관리할 때 훅을 사용한다. 훅이란 함수형 컴포넌트에서 상태(state)와 생명주기(lifecycle)를 쉽게 관리할 수 있도록 도와주는 리엑트에서 제공하는 특별한 함수이다.

useState(), useEffect(), useRef() 등이 있다.

const [state, setState] = useState(0)

- state: 상태 값을 저장하는 변수로 보통 상태 변수라고 부른다.
- setState: 상태를 변경하는 함수로, 상태 변경 함수라고 부른다.

- 예) count 상태를 변경하는 코드

```
const [count, setCount] = useState(0)
```

컴포넌트(Component) 상태

- **Counter 구현**

Counter

Counter 컴포넌트입니다.

현재 카운트: 5

증가

컴포넌트(Component) 상태

- **useState() – Counter.jsx**

```
import React, { useState } from 'react';

const Counter = () => {
  // count 상태와 setCount 함수를 useState 훅을 사용하여 정의
  const [count, setCount] = useState(0);

  // 증가 버튼 클릭 시 호출되는 함수
  const increment = () => {
    setCount(count + 1);
  };
```

컴포넌트(Component) 상태

- **useState() – Counter.jsx**

```
  return (
    <div>
      <h2>Counter</h2>
      <p>Counter 컴포넌트입니다.</p>
      <p>현재 카운트: {count}</p>
      <button onClick={increment}>증가</button>
    </div>
  );
}

export default Counter;
```

```
- 인라인 핸들러 함수
<button onClick={() => setCount(count + 1)}>증가</button>
```

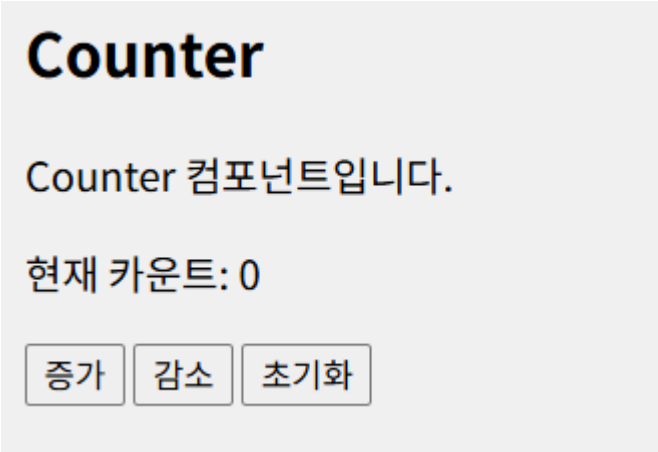
실습 문제

- **감소 버튼과 초기화 버튼 만들기**

  - 요구 사항

    감소 버튼을 클릭하면 현재 카운트가 1감소하고,

    초기화 버튼을 클릭하면 현재 카운트가 0이 됩니다.



useState() 함수

- 음료 추가 – 배열 상태 관리

음료 추가

음료 추가 컴포넌트입니다.

현재 음료: 쥬스, 콜라, 커피

음료 추가

useState() 함수

- 음료 추가 – 배열 상태 관리

```
import { useState } from 'react';

const AddDrink = () => {
  const [drinkName, setDrinkName] = useState(['쥬스', '콜라']);

  const addDrink = () => {
    // 기존의 drinkName 배열에 '커피'를 추가하여 새로운 배열을 생성
    setDrinkName([...drinkName, '커피']);
  };

  return (
    <div>
      <h2>음료 추가</h2>
      <p>음료 추가 컴포넌트입니다.</p>
      <p>현재 음료: {drinkName.join(', ')}</p>
      <button onClick={addDrink}>음료 추가</button>
    </div>
  );
}
```

useState() 함수

- 자동차 연식 업데이트

자동차 컴포넌트

브랜드: 현대

모델: 쏘나타

연식: 2025

연식 업데이트

useState() 함수

- **자동차 연식 업데이트**

```
const Car = () => {
  const [car, setCar] = useState({
    brand: '현대',
    model: '쏘나타',
    year: 2020
  });

  const updateYear = () => {
    /* car 객체의 year 속성만 업데이트하기 위해
       기존의 car 객체를 복사하고 year 속성만 변경 */
    setCar({ ...car, year: 2025 });
  };

  return (
    <div>
      <h2>자동차 컴포넌트</h2>
      <p>브랜드: {car.brand}</p>
      <p>모델: {car.model}</p>
      <p>연식: {car.year}</p>
      <button onClick={updateYear}>연식 업데이트</button>
    </div>
  );
}
```

상품 관리 시스템 구현

```
# 📦 상품 관리 시스템 - 작업지시서

## 🎯 주요 기능

### 1. 상품 추가 (Add Product)
- **입력 값**: 상품명 (text input)
- **유효성 검사**: 빈 값 입력 방지
- **고유 ID**: `Date.now()`를 사용하여 자동 생성
- **동작**: 입력 후 "추가" 버튼 클릭 시 상품 목록에 추가
- **결과**: 콘솔에 현재 상품 목록 배열 출력

### 2. 상품 목록 표시 (Product List Display)
- **형식**: 비순차 리스트 (`<ul>`)
- **표시 내용**: 상품명 + 삭제 버튼
- **빈 목록 처리**: "추가된 상품이 없습니다." 메시지 표시

### 3. 상품 삭제 (Delete Product)
- **방법**: 각 상품 항목의 "삭제" 버튼 클릭
- **동작**: `filter()` 메서드로 해당 상품만 제외
- **결과**: 콘솔에 업데이트된 상품 목록 배열 출력
```

useState() 함수

- 상품 입력 받고 추가하기

useState() 함수

- 상품 입력 받고 추가하기

```react_app/src/AddProduct.jsx
import { useState } from 'react';

const AddProduct = () => {
  // productName 상태와 setProductName 함수를 useState 훅을 사용
  const [productName, setProductName] = useState('');
  const [products, setProducts] = useState([]);

  const handleInputChange = (event) => {
    // console.log(event);
    // 입력된 상품 이름을 상태로 업데이트
    setProductName(event.target.value);
  };
```

```
▸ isDefaultPrevented: ƒ functionThatR
▸ isPropagationStopped: ƒ functionTha
  isTrusted: true
▸ nativeEvent: InputEvent {isTrusted:
▾ target: input
    value: "키보드"
  ▸ __reactEvents$ls71p5mwkjp: Set(1)
```

useState() 함수

- **상품 입력 받고 추가하기**

```
const handleAddProduct = () => {
  // 상품 추가 로직을 여기에 작성
  if (productName.trim() === '') {
    alert('상품 이름을 입력해주세요.');
    return;
  }
  //id는 고유한 값이 필요하므로 Date.now()를 사용하여 생성
  const newProducts = [...products, { id: Date.now(), name: productName }];
  console.log('상품 목록:', newProducts);
  setProducts(newProducts);
  setProductName(''); // 입력 필드 초기화
};
```

```
상품 목록:                                  AddProduct2.jsx?t=
  ▾ [{…}]
    ▸ 0: {id: 1773887342049, name: '키보드'}
      length: 1
    ▸ [[Prototype]]: Array(0)
상품 목록:                                  AddProduct2.jsx?t=
  ▾ (2) [{…}, {…}]
    ▸ 0: {id: 1773887342049, name: '키보드'}
    ▸ 1: {id: 1773887348826, name: '마우스'}
      length: 2
    ▸ [[Prototype]]: Array(0)
```

useState() 함수

- **상품 입력 받고 추가하기**

```
  return (
    <div>
      <h2>상품 추가</h2>
      <p>상품 추가 컴포넌트입니다.</p>
      <input
        type="text"
        value={productName}
        onChange={handleInputChange}
        placeholder="상품 이름을 입력하세요"
      />
      <button onClick={handleAddProduct}>추가</button>
    </div>
  );
}

export default AddProduct;
```

useState() 함수

- 상품 출력 및 삭제 기능

useState() 함수

- **상품 출력 및 삭제 기능**

```
const handleDeleteProduct = (id) => {
  // 상품 삭제 로직을 여기에 작성
  // id를 기준으로 해당 상품을 제외한 새로운 배열을 생성하여 상태 업데이트
  setProducts(products.filter(product => product.id !== id));
};
```

useState() 함수

- 상품 출력 및 삭제 기능

```
{/* 추가된 상품 목록 */}
<div style={{ marginTop: '20px' }}>
  <h3>추가된 상품 목록</h3>
  {products.length === 0 ? (
    <p>추가된 상품이 없습니다.</p>
  ) : (
    <ul>
      {products.map((product) => (
        // key는 id를 사용하여 각 상품을 식별
        <li key={product.id}>
          {product.name}{' '}
          <button
            onClick={() => handleDeleteProduct(product.id)}
            style={{ marginLeft: '10px', color: 'red' }}
          >
            삭제
          </button>
        </li>
      ))}
    </ul>
  )}
</div>
```